

COMP 421 - Database Systems

Project 2: Creating and Working with Your Database

Group 99: Luca Mitran, Nicholas Imperioli, Ping-Chieh Tu

March 10th 2025

1 Relational Schema

```
Developer(did,
          dname NOT NULL,
          demail NOT NULL,
          dphone,
          PRIMARY KEY (did))
Publisher(pid,
          pname NOT NULL,
          pemail NOT NULL,
          pphone,
          PRIMARY KEY (pid))
Game(gid,
     date NOT NULL,
     rating,
     price NOT NULL,
     gname NOT NULL,
     PRIMARY KEY (gid))
Category(name,
         PRIMARY KEY (cname))
Users(uid,
      uemail NOT NULL,
      password NOT NULL,
      PRIMARY KEY (uid))
BillingInfo(uid,
            address,
            uname NOT NULL,
            payMethod NOT NULL,
            PRIMARY KEY (uid),
            FOREIGN KEY (uid) REFERENCES Users)
```

```

Transactions(tid,
            date NOT NULL,
            amount NOT NULL,
            status NOT NULL,
            uid NOT NULL,
            PRIMARY KEY (tid),
            FOREIGN KEY (uid) REFERENCES Users)

Discount(gid, startDate,
        endDate NOT NULL,
        percentOff NOT NULL,
        PRIMARY KEY (gid, startDate),
        FOREIGN KEY (gid) REFERENCES Game)

MakePublish(gid, did NOT NULL, pid NOT NULL,
            PRIMARY KEY (gid),
            FOREIGN KEY (gid) REFERENCES Game,
            FOREIGN KEY (did) REFERENCES Developer,
            FOREIGN KEY (pid) REFERENCES Publisher)

Wish(uid, gid,
    PRIMARY KEY (uid, gid),
    FOREIGN KEY (uid) REFERENCES Users,
    FOREIGN KEY (gid) REFERENCES Game)

Own(uid, gid,
    PRIMARY KEY (uid, gid),
    FOREIGN KEY (uid) REFERENCES Users,
    FOREIGN KEY (gid) REFERENCES Game)

InCart(uid, gid,
    PRIMARY KEY (uid, gid),
    FOREIGN KEY (uid) REFERENCES Users,
    FOREIGN KEY (gid) REFERENCES Game)

Friend(uid1, uid2,
    status NOT NULL,
    PRIMARY KEY (uid1, uid2),
    FOREIGN KEY (uid1) REFERENCES Users,
    FOREIGN KEY (uid2) REFERENCES Users)

BelongsTo(gid, cname,
    PRIMARY KEY (gid, cname),
    FOREIGN KEY (gid) REFERENCES Game,
    FOREIGN KEY (cname) REFERENCES Category)

Buy(uid, gid, tid,
    PRIMARY KEY (uid, gid, tid),
    FOREIGN KEY (uid) REFERENCES Users,
    FOREIGN KEY (gid) REFERENCES Game,
    FOREIGN KEY (tid) REFERENCES Transactions)

```

2 Pending Constraints

1. Overlapping Discount Dates:

For a given game (`gid`), discount periods (e.g., `startDate` and `endDate`) shouldn't overlap. Since a game cannot have multiple discounts at the same time, application-level logic can be implemented later to enforce this new constraint.

2. Transaction Consistency:

To make sure a user doesn't rebuy a game they own, a transaction (`tid`) in the `Buy` table should directly reference a game (`gid`) that the user (`uid`) doesn't already own. Using application-level logic to check the `Own` table before allowing a transaction to be recorded in the `Buy` table should be implemented.

3. Friend Relationship Ordering/Reflexivity:

The current constraint `CHECK (uid1 != uid2)` only prevents a user from friending themselves, it doesn't prevent duplicate entries for example when `uid1` and `uid2` are swapped (e.g., (1, 2) and (2, 1)). Implementing application-level logic allows for only one of the 2 orderings to be possible.

4. Game Ownership After Purchase:

When a user (`uid`) buys a game (`gid`), the game should be added to the `Own` table automatically. Use application-level logic to automatically do this.

3 SQL Queries

3.1 Query 1

We would like to know the game name, release date, developer name, publisher name, number of sales as `sales` of the best selling games based on the number of people that bought them.

SQL query:

```
SELECT g.gname AS game_name,
       g.date AS release_date,
       d.dname AS developer_name,
       p.pname AS publisher_name,
       COUNT(b.uid) AS sales
FROM Buy b
JOIN Game g ON g.gid = b.gid
JOIN Transactions t ON b.tid = t.tid
JOIN MakePublish m ON m.gid = g.gid
JOIN Developer d ON m.did = d.did
JOIN Publisher p ON m.pid = p.pid
WHERE t.status = 'Completed'
GROUP BY b.gid, g.gname, g.date, d.dname, p.pname
HAVING COUNT(b.uid) >= ALL (SELECT COUNT(uid) FROM Buy GROUP BY gid);
```

```

cs421g99@winter2025-comp421:~/project2/oscar$ cat sql1.sql
SELECT g.gname AS game_name,
       g.date AS release_date,
       d.dname AS developer_name,
       p.pname AS publisher_name,
       COUNT(b.uid) AS sales
FROM Buy b
JOIN Game g ON g.gid = b.gid
JOIN Transactions t ON t.tid = b.tid
JOIN MakePublish m ON m.gid = g.gid
JOIN Developer d ON d.did = m.did
JOIN Publisher p ON p.pid = m.pid
WHERE t.status = 'Completed'
GROUP BY b.gid, g.gname, g.date, d.dname, p.pname
HAVING COUNT(b.uid) >= ALL (SELECT COUNT(uid) FROM Buy GROUP BY gid);
cs421g99@winter2025-comp421:~/project2/oscar$ db2 -t -f sql1.sql

```

GAME_NAME	RELEASE_DATE	DEVELOPER_NAME	PUBLISHER_NAME	SALES
Oxygen Not Included	07/30/2019	Klei Entertainment	Klei Entertainment	2

1 record(s) selected.

3.2 Query 2

We would like to find the best selling game category of each year
SQL query:

```

WITH CategorySales AS (
    SELECT YEAR(t.date) as year, bt.cname, COUNT(b.uid) AS sales
    FROM Buy b
    JOIN Transactions t ON t.tid = b.tid
    JOIN BelongsTo bt ON bt.gid = b.gid
    WHERE status = 'Completed'
    GROUP BY YEAR(t.date), bt.cname
),
Ranks AS (
    SELECT year, cname, sales,
           RANK() OVER(
               PARTITION BY year
               ORDER BY sales DESC
           ) as rank
    FROM CategorySales
)
SELECT year, cname FROM Ranks
WHERE rank = 1;

```

```

cs421g99@winter2025-comp421:~/project2/oscar$ cat sql2.sql
WITH CategorySales AS (
    SELECT YEAR(t.date) as year, bt.cname, COUNT(b.uid) AS sales
    FROM Buy b
    JOIN Transactions t ON t.tid = b.tid
    JOIN BelongsTo bt ON bt.gid = b.gid
    WHERE status = 'Completed'
    GROUP BY YEAR(t.date), bt.cname
),
Ranks AS (
    SELECT year, cname, sales,
           RANK() OVER(
               PARTITION BY year
               ORDER BY sales DESC
           ) as rank
    FROM CategorySales
)
SELECT year, cname FROM Ranks
WHERE rank = 1;
cs421g99@winter2025-comp421:~/project2/oscar$ db2 -t -f sql2.sql

YEAR      CNAME
-----
2023 Indie

1 record(s) selected.

```

3.3 Query 3

We each user to receive a list of what their friends last bought game was as a list of game recommendations.

SQL query:

```
WITH FriendWith (user, friend) AS (  
    SELECT uid1 AS user, uid2 AS friend  
    FROM Friend  
    WHERE status = 'Accepted'  
    UNION  
    SELECT uid2 AS user, uid1 AS friend  
    FROM Friend  
    WHERE status = 'Accepted'  
),  
LastBought (user, game_name, rank) AS (  
    SELECT b.uid AS user, g.gname AS game_name,  
    RANK () OVER (  
        PARTITION BY b.uid  
        ORDER BY t.date DESC  
    ) as rank  
    FROM Buy b  
    JOIN Game g ON b.gid = g.gid  
    JOIN Transactions t ON b.tid = t.tid  
    WHERE t.status = 'Completed'  
)  
SELECT u.uid, COALESCE(lb.game_name, 'None') AS recommendations  
FROM Users u  
LEFT JOIN FriendWith fw ON u.uid = fw.user  
LEFT JOIN LastBought lb ON fw.friend = lb.user AND lb.rank = 1  
ORDER BY u.uid;
```

```
cs421g59@winter2025-comp421:~/project2/oscar$ cat sql3.sql  
WITH FriendWith (user, friend) AS (  
    SELECT uid1 AS user, uid2 AS friend  
    FROM Friend  
    WHERE status = 'Accepted'  
    UNION  
    SELECT uid2 AS user, uid1 AS friend  
    FROM Friend  
    WHERE status = 'Accepted'  
),  
LastBought (user, game_name, rank) AS (  
    SELECT b.uid AS user, g.gname AS game_name,  
    RANK () OVER (  
        PARTITION BY b.uid  
        ORDER BY t.date DESC  
    ) as rank  
    FROM Buy b  
    JOIN Game g ON b.gid = g.gid  
    JOIN Transactions t ON b.tid = t.tid  
    WHERE t.status = 'Completed'  
)  
SELECT u.uid, COALESCE(lb.game_name, 'None') AS recommendations  
FROM Users u  
LEFT JOIN FriendWith fw ON u.uid = fw.user  
LEFT JOIN LastBought lb ON fw.friend = lb.user AND lb.rank = 1  
ORDER BY u.uid;  
cs421g59@winter2025-comp421:~/project2/oscar$ db2 -t -f sql3.sql  
  
UID          RECOMMENDATIONS  
-----  
1 None  
2 None  
3 DogFighter  
4 Kor  
5 None  
6 Hero of the Kingdom II  
7 None  
8 None  
9 Slot Shots Pinball Collection  
10 Cthulhu Realms  
  
10 record(s) selected.
```

3.4 Query 4

We would like to give each user a recommended game category based on the games on their wishlist. SQL query:

```
WITH WishCategory AS (  
    SELECT u.uid, bt.cname, COUNT(bt.cname) as count  
    FROM Users u  
    LEFT JOIN Wish w ON w.uid = u.uid  
    JOIN BelongsTo bt ON w.gid = bt.gid  
    GROUP BY u.uid, bt.cname  
) ,  
Ranks AS (  
    SELECT uid, cname,  
    RANK () OVER (  
        PARTITION BY uid  
        ORDER BY count DESC, cname ASC  
    ) as rank  
    FROM WishCategory  
)  
SELECT u.uid, COALESCE(r.cname, 'None') AS recommendations  
FROM Users u  
LEFT JOIN Ranks r ON u.uid = r.uid AND r.rank = 1  
ORDER BY u.uid;
```

```
cs421g99@winter2025-comp421:~/project2/oscar$ cat sql4.sql  
WITH WishCategory AS (  
    SELECT u.uid, bt.cname, COUNT(bt.cname) as count  
    FROM Users u  
    LEFT JOIN Wish w ON w.uid = u.uid  
    JOIN BelongsTo bt ON w.gid = bt.gid  
    GROUP BY u.uid, bt.cname  
) ,  
Ranks AS (  
    SELECT uid, cname,  
    RANK () OVER (  
        PARTITION BY uid  
        ORDER BY count DESC, cname ASC  
    ) as rank  
    FROM WishCategory  
)  
SELECT u.uid, COALESCE(r.cname, 'None') AS recommendations  
FROM Users u  
LEFT JOIN Ranks r ON u.uid = r.uid AND r.rank = 1  
ORDER BY u.uid;  
cs421g99@winter2025-comp421:~/project2/oscar$ db2 -t -f sql4.sql  
  
UID          RECOMMENDATIONS  
-----  
1 Action  
2 Action  
3 Adventure  
4 Indie  
5 Casual  
6 Action  
7 Action  
8 Indie  
9 Casual  
10 Adventure  
  
10 record(s) selected.
```

3.5 Query 5

We would like to give each user a list of games that their friends have on their wishlist SQL query:

```
WITH FriendWith (user, friend) AS (  
    SELECT uid1 AS user, uid2 AS friend
```

```

FROM Friend
WHERE status = 'Accepted'
UNION
SELECT uid2 AS user, uid1 AS friend
FROM Friend
WHERE status = 'Accepted'
)
SELECT fw.user, fw.friend, g.gname AS wish_game
FROM FriendWith fw
JOIN Wish w ON w.uid = fw.friend
JOIN Game g ON w.gid = g.gid
ORDER BY fw.user;

```

```

cs421g99@winter2025-comp421:~/project2/oscar$ cat sql5.sql
WITH FriendWith (user, friend) AS (
  SELECT uid1 AS user, uid2 AS friend
  FROM Friend
  WHERE status = 'Accepted'
  UNION
  SELECT uid2 AS user, uid1 AS friend
  FROM Friend
  WHERE status = 'Accepted'
)
SELECT fw.user, fw.friend, g.gname AS wish_game
FROM FriendWith fw
JOIN Wish w ON w.uid = fw.friend
JOIN Game g ON w.gid = g.gid
ORDER BY fw.user;
cs421g99@winter2025-comp421:~/project2/oscar$ db2 -t -f sql5.sql

```

USER	FRIEND	WISH_GAME
3	4	Steam Squad
3	4	Endline
4	3	Deadfall Adventures
4	3	Kor
5	6	Aurion: Legacy of the Kori-Odan
6	5	Hero of the Kingdom II
6	5	Slot Shots Pinball Collection
9	10	Sally Face - Episode One
10	9	Fat City

```

9 record(s) selected.

```

4 SQL Modification

4.1 Mod 1

There is a 40% discount for 7 days on all games in the Action category that have never had a discount before.

SQL query:

```

INSERT INTO Discount (gid, startDate, endDate, percentOff)
SELECT b.gid,
       CURRENT_DATE as startDate,
       CURRENT_DATE + 7 as endDate,
       40 as percentOff
FROM BelongsTo b
WHERE b.cname = 'Action'
AND NOT EXISTS (

```

```

SELECT * FROM Discount d WHERE d.gid = b.gid
);

```

```

cs421g99@winter2025-comp421:~/project2/oscar$ cat mod1.sql
INSERT INTO Discount (gid, startDate, endDate, percentOff)
SELECT b.gid,
       CURRENT_DATE as startDate,
       CURRENT_DATE + 7 as endDate,
       40 as percentOff
FROM BelongsTo b
WHERE b.cname = 'Action'
AND NOT EXISTS (
    SELECT * FROM Discount d WHERE d.gid = b.gid
);
cs421g99@winter2025-comp421:~/project2/oscar$ db2 -t -f mod1.sql
DB20000I The SQL command completed successfully.

```

4.2 Mod 2

The Company Rockstar Games is having a price increase of 10\$ on all of their games.
SQL query:

```

UPDATE Game g
SET g.price = g.price + 10
WHERE g.gid IN (
    SELECT mp.gid
    FROM MakePublish mp
    JOIN Publisher p ON p.pid = mp.pid
    WHERE p.pname = 'Rockstar Games'
);

```

```

cs421g99@winter2025-comp421:~/project2/oscar$ cat mod2.sql
UPDATE Game g
SET g.price = g.price + 10
WHERE g.gid IN (
    SELECT mp.gid
    FROM MakePublish mp
    JOIN Publisher p ON p.pid = mp.pid
    WHERE p.pname = 'Rockstar Games'
);
cs421g99@winter2025-comp421:~/project2/oscar$ db2 -t -f mod2.sql
DB20000I The SQL command completed successfully.

```

5 Views

5.1 View 1

BasicGameInfo is a simple view that selects budget games from the Game table, specifically those priced under \$20.00. This view provides easy access to affordable games in the marketplace, which is useful for customers looking for budget options.

SQL Query:


```

CREATE VIEW
BasicGameInfo AS
SELECT gid, gname, date, price
FROM Game
WHERE price < 20.00;

```

```

cs421g99@winter2025-comp421:~/project2/luca$ cat view1.sql
CREATE VIEW BasicGameInfo AS
SELECT gid, gname, date, price
FROM Game
WHERE price < 20.00;
cs421g99@winter2025-comp421:~/project2/luca$ db2 -tvf view1.sql
CREATE VIEW BasicGameInfo AS SELECT gid, gname, date, price FROM Game WHERE price < 20.00
DB20000I The SQL command completed successfully.

cs421g99@winter2025-comp421:~/project2/luca$ db2 "SELECT * FROM BasicGameInfo FETCH FIRST 5 ROWS ONLY"

GID      GNAME                                     DATE      PRICE
-----
12140 Max Payne                           01/06/2011    3.49
20200 Galactic Bowling                   10/21/2008   19.99
22670 Alien Breed 3: Descent              11/17/2010    9.99
42500 DogFighter                         06/14/2010    0.99
231330 Deadfall Adventures                11/15/2013   19.99

5 record(s) selected.

cs421g99@winter2025-comp421:~/project2/luca$ db2 "INSERT INTO BasicGameInfo (gid, gname, date, price) VALUES (999999, 'New Budget Game', CURRENT DATE, 15.99)"
DB20000I The SQL command completed successfully.

```

The insert operation on this view is successful because BasicGameInfo is a simple view over a single table (Game) with no complex joins or aggregations. The insert operation maps directly to an insert on the underlying Game table, and all necessary columns have valid values.

5.2 View 2

GamePurchaseDetails is a complex view that joins multiple tables (Game, Buy, Transactions, and Users) to provide comprehensive information about game purchases. It includes game details, purchase dates, payment amounts, and buyer information for completed transactions.

SQL Query:

```

CREATE VIEW GamePurchaseDetails AS
SELECT
g. gid,
g. gname, g-price,
t.date AS purchase_date,
t. amount AS paid_amount,
u. uid,
u. uemail AS buyer_email
FROM
Game g
JOIN
Buy b ON g-gid = b.gid
JOIN
Transactions t ON b.tid = t.tid
JOIN
Users u ON b.uid = u.uid
WHERE
t. status = 'Completed';

```

```

cs421g99@winter2025-comp421:~/project2/luca$ cat view2.sql
CREATE VIEW GamePurchaseDetails AS
SELECT
  g.gid,
  g.gname,
  g.price,
  t.date AS purchase_date,
  t.amount AS paid_amount,
  u.uid,
  u.uemail AS buyer_email
FROM
  Game g
JOIN
  Buy b ON g.gid = b.gid
JOIN
  Transactions t ON b.tid = t.tid
JOIN
  Users u ON b.uid = u.uid
WHERE
  t.status = 'Completed';
cs421g99@winter2025-comp421:~/project2/luca$ db2 -tvf view2.sql
CREATE VIEW GamePurchaseDetails AS SELECT g.gid, g.gname, g.price, t.date AS purchase_date, t.amount AS paid_amount, u.uid, u.uemail AS buyer_email FROM Game g JOIN Buy b ON g.gid = b.gid JO
IN Transactions t ON b.tid = t.tid JOIN Users u ON b.uid = u.uid WHERE t.status = 'Completed'
DB20000I The SQL command completed successfully.

cs421g99@winter2025-comp421:~/project2/luca$ db2 "SELECT * FROM GamePurchaseDetails FETCH FIRST 5 ROWS ONLY"

  GID          GNAME                                     PRICE    PURCHASE_DATE PAID_AMOUNT  UID      BUYER_EMAIL
-----
12140 Max Payne                                     3.49 10/01/2023         0.00      1 alice.jones@gmail.com
42500 DogFighter                                    0.99 09/10/2023         0.89      4 diana.wilson@gmail.com
346560 Hero of the Kingdom II                       7.99 10/05/2023        14.99      5 edward.moore@hotmail.com
457140 Oxygen Not Included                         24.99 08/15/2023        14.99      7 george.white@gmail.com
457140 Oxygen Not Included                         24.99 09/25/2023         9.74      8 hannah.green@hotmail.com

5 record(s) selected.

cs421g99@winter2025-comp421:~/project2/luca$ db2 "INSERT INTO GamePurchaseDetails (gid, gname, price, purchase_date, paid_amount, uid, buyer_email) VALUES (999999, 'Test Game', 29.99, CURREN
T DATE, 29.99, 1, 'alice.jones@gmail.com')"
```

The insert operation on this view fails with an SQL0150N error because GamePurchaseDetails is a complex view that joins multiple tables. According to DB2's rules for view insertability, inserts are not permitted on views that include joins, as the database cannot automatically determine how to distribute the inserted data across the underlying tables.

6 Check Constraints

6.1 Check 1

This constraint ensures that discount percentages in the Discount table are always greater than 0 and less than or equal to 100%. This is a critical business rule for the video game marketplace as it prevents invalid discount values that could cause pricing errors.

SQL Query:

```

ALTER TABLE Discount
ADD CONSTRAINT check_discount_range
CHECK (percentoff > 0 AND percentoff <= 100);
```

When a user attempts to insert a discount percentage that exceeds 100%, the database rejects the operation and returns an SQL0545N error indicating that the row does not satisfy the check constraint.

```

cs421g99@winter2025-comp421:~/project2/luca$ cat check1.sql
ALTER TABLE Discount
ADD CONSTRAINT check_discount_range
CHECK (percentOff > 0 AND percentOff <= 100);
cs421g99@winter2025-comp421:~/project2/luca$ db2 -tvf check1.sql
ALTER TABLE Discount ADD CONSTRAINT check_discount_range CHECK (percentOff > 0 AND percentOff <= 100)
DB20000I The SQL command completed successfully.

cs421g99@winter2025-comp421:~/project2/luca$ db2 "INSERT INTO Discount (gid, startDate, endDate, percentOff) VALUES (12140, '2024-03-01', '2024-03-31', 25.00)"
DB20000I The SQL command completed successfully.
cs421g99@winter2025-comp421:~/project2/luca$ db2 "INSERT INTO Discount (gid, startDate, endDate, percentOff) VALUES (12140, '2024-04-01', '2024-04-30', 120.00)"
DB21034E The command was processed as an SQL statement because it was not a
valid Command Line Processor command. During SQL processing it returned:
SQL0545N The requested operation is not allowed because a row does not
satisfy the check constraint "CS421G99.DISCOUNT.CHECK_DISCOUNT_RANGE".
SQLSTATE=23513

```

6.2 Check 2

This constraint restricts the status field in the Friend table to only contain one of the predefined values: Pending, Accepted, Blocked, or Rejected. This maintains data consistency in the friend relationship system and prevents invalid status values.

SQL Query:

```

ALTER TABLE Friend
ADD CONSTRAINT check_friend_status
CHECK (status IN ('Pending',
'Accepted', 'Blocked',
'Rejected'));

```

```

cs421g99@winter2025-comp421:~/project2/luca$ cat check2.sql
ALTER TABLE Friend
ADD CONSTRAINT check_friend_status
CHECK (status IN ('Pending', 'Accepted', 'Blocked', 'Rejected'));
cs421g99@winter2025-comp421:~/project2/luca$ db2 -tvf check2.sql
ALTER TABLE Friend ADD CONSTRAINT check_friend_status CHECK (status IN ('Pending', 'Accepted', 'Blocked', 'Rejected'))
DB20000I The SQL command completed successfully.

cs421g99@winter2025-comp421:~/project2/luca$ db2 "INSERT INTO Friend (uid1, uid2, status) VALUES (1, 3, 'Pending')"
DB20000I The SQL command completed successfully.
cs421g99@winter2025-comp421:~/project2/luca$ db2 "INSERT INTO Friend (uid1, uid2, status) VALUES (2, 4, 'Maybe')"
DB21034E The command was processed as an SQL statement because it was not a
valid Command Line Processor command. During SQL processing it returned:
SQL0545N The requested operation is not allowed because a row does not
satisfy the check constraint "CS421G99.FRIEND.CHECK_FRIEND_STATUS".
SQLSTATE=23513

```

The constraint successfully prevents users from entering undefined status values in the Friend table, returning an SQL0545N error when attempting to use 'Maybe' as a status, which is not in the allowed list.

7 Creativity

7.1 Real data sets

We used the dataset found on Kaggle¹ to generate the data for 100 games. `loaddata.py` shows how the data was scraped from the database and the outputs are also provided in the folder **Scraped Data**. The size of the raw data file is too large to include in the submission as it contains 1000s of games; Using the link in the footnote will allow you to access the data.

¹<https://www.kaggle.com/datasets/fronkongames/steam-games-dataset>

7.2 Advanced SQL Features

In SQL 2, SQL 3 and SQL 4, we used `RANK() OVER ([partition] ORDER BY order) AS rank`. This will rank the tables by order (from 1 to n), the partition is like `GROUP BY`. In SQL 2, it groups the sales by year and rank in each year. In SQL 3, it groups the games bought and rank for each user by the time they bought it. In SQL 4, it groups the games category and rank for each user by the count of the category in their wishlist. The function `COALESCE(...)` will return the first non-null element in the list. For example in SQL 3, `COALESCE(lb.game_name, 'None')` returns `lb.game_name` and if it is not NULL, and will return `'None'` if `lb.game_name` is NULL.

8 Teamwork

We communicated regularly during the project's availability with meetings at least once a week. The work was divided evenly among us where one person worked on parts 3 and 4, one worked on parts 5 and 6, and the last person worked on parts 7 and 8. Part 2 was a copy paste of the relational schema from deliverable 1 as nothing changed there and the creativity section was a joint effort between all members. After a section was completed, we would join a meeting and discuss any changes/challenges that were present and looked towards solving them. Once every section was complete we had a final meeting verifying that all requirements were met and that the work was to an acceptable standard.